

EcoFleet AI: Intelligent Electric Vehicle Fleet Monitoring and Route Optimization Platform

Submitted in the partial fulfilment for the Course of

CSA1016 SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE WITH AI

Submitted by

KENISHA JUDITH J (192572071)

Under the Supervision of

Dr.K.ANITADAVAMANI



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

1. Introduction

The transportation industry is increasingly adopting electric vehicles (EVs) to reduce carbon emissions, fuel dependency, and operational costs. However, managing an EV fleet requires continuous monitoring of battery health, energy consumption, vehicle availability, charging requirements, and route efficiency.

EcoFleet AI is an intelligent electric vehicle fleet monitoring and route optimization platform designed to support fleet managers through AI-based analytics, IoT data monitoring, predictive maintenance, charging optimization, and energy-efficient route recommendations.

Since the platform handles continuously changing vehicle and operational data, reliable software delivery is essential. A Continuous Integration and Continuous Deployment (CI/CD) pipeline is therefore proposed to automate code integration, building, testing, quality analysis, security verification, packaging, and deployment.

The CI/CD design follows the assessment requirement of covering source-code management, build, testing, quality checking, packaging, deployment, security, notifications, and rollback mechanisms.

2. Project Overview

EcoFleet AI consists of several major functional components:

- **Fleet Monitoring Module** – monitors vehicle status, battery level, energy consumption, and availability.
- **Battery Prediction Module** – analyzes battery information and predicts degradation.
- **Predictive Maintenance Module** – identifies possible maintenance requirements.
- **Charging Optimization Module** – recommends suitable charging schedules.
- **Route Optimization Module** – recommends energy-efficient routes using vehicle and route information.

- **Fleet Dashboard** – provides fleet managers with real-time operational information.
- **Analytics and Reporting Module** – generates reports on energy efficiency, fleet utilization, and operational performance.

The system can receive simulated or IoT-generated vehicle data and process it through backend services and AI models.

3. CI/CD Requirements Analysis

The EcoFleet AI platform requires a CI/CD pipeline because multiple software components may be developed and updated independently. Changes to the backend, AI models, dashboard, or database must be tested before deployment.

The major CI/CD requirements are:

1. **Version_Control**

All source code must be maintained in a Git repository.

2. **Automated_Build**

Every approved code change should automatically trigger a build process.

3. **Automated_Testing**

Unit, integration, API, and basic acceptance tests should be executed automatically.

4. **Code_Quality_Analysis**

Source code should be checked for bugs, code smells, and maintainability issues.

5. **Security_Testing**

Dependencies and Python source code should be scanned for security vulnerabilities.

6. **Containerized_Packaging**

The application should be packaged using Docker to provide a consistent execution environment.

7. **Staging_Deployment**

Successful builds should first be deployed to a staging environment for validation.

8. Production_Deployment

Only validated builds should be deployed to production.

9. Monitoring

The deployed system should be monitored for failures and performance problems.

10. Rollback

If a production deployment causes problems, the previous stable version should be restored.

4. Proposed CI/CD Pipeline Architecture

The proposed EcoFleet AI pipeline consists of the following stages:

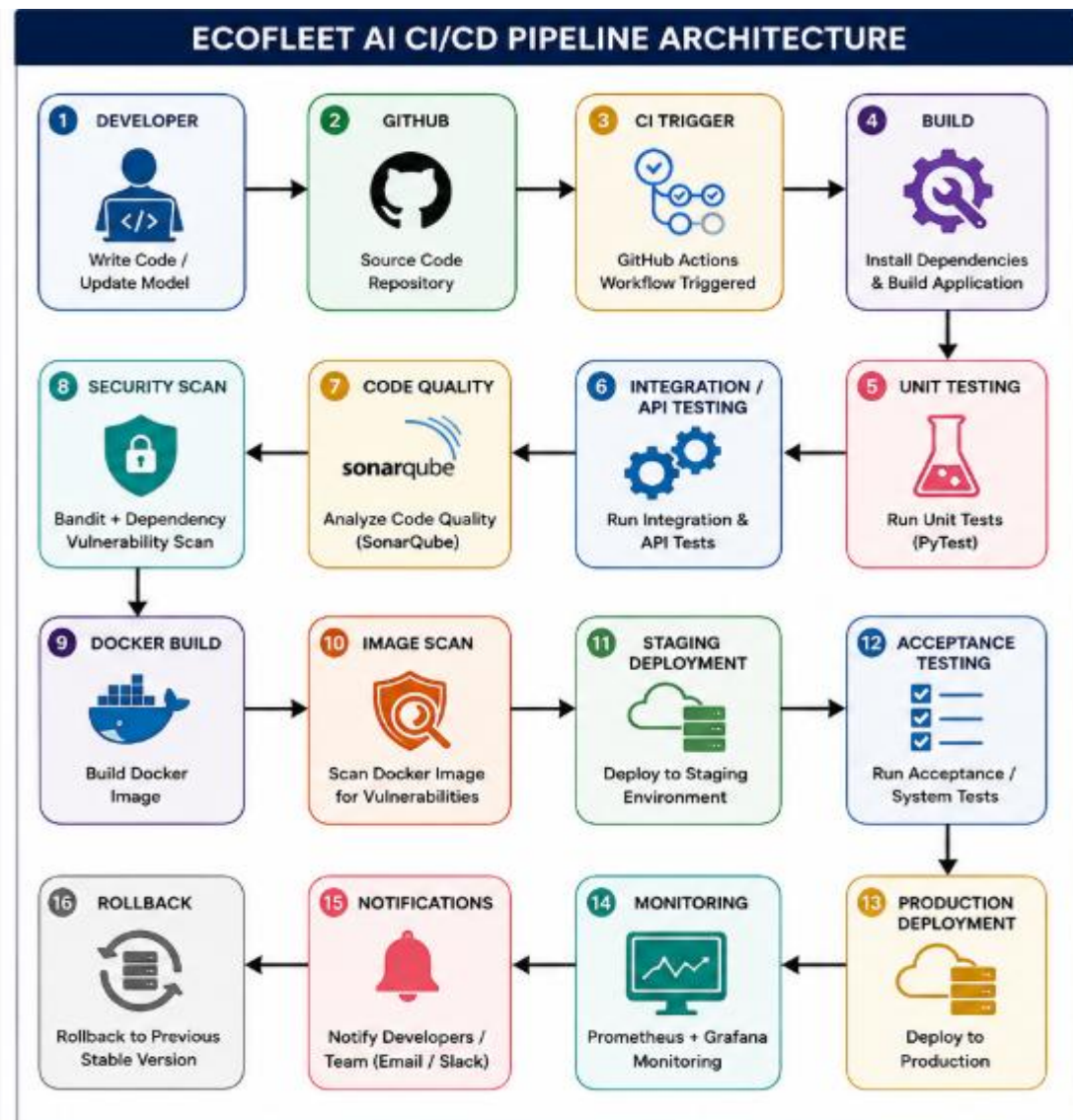


Figure 1: Proposed CI/CD Pipeline Architecture for EcoFleet AI

Code → Commit → Build → Test → Quality Check → Security Scan →
Docker Package → Image Scan → Staging → Acceptance Test → Production
→ Monitoring

5. Pipeline Stages and Workflow

5.1 Source Code Management

The project source code will be maintained in a GitHub repository.

The repository may contain:

EcoFleet-AI

```
|  
|— backend/  
|— frontend/  
|— ml_models/  
|— tests/  
|— database/  
|— Dockerfile  
|— docker-compose.yml  
|— requirements.txt  
└─ .github/  
    └─ workflows/  
        └─ cicd.yml
```

Developers work on separate branches and submit pull requests before merging changes into the main branch.

Purpose

- Maintain version history.
- Support collaborative development.
- Track changes.
- Enable pull-request-based review.
- Provide the trigger for the CI/CD pipeline.

6. Build Stage

After a developer pushes code or creates a pull request, GitHub Actions automatically starts the pipeline.

The build stage performs:

1. Checkout of the latest source code.
2. Setup of the Python environment.
3. Installation of dependencies.
4. Validation of project structure.
5. Application build preparation.

For example, the required Python packages can be installed using:

```
pip install -r requirements.txt
```

If the project contains frontend components, the frontend dependencies can also be installed and built during this stage.

Objective

The build stage ensures that the application can be prepared successfully before testing begins.

7. Automated Testing Strategy

Automated testing is an important part of the EcoFleet AI CI/CD pipeline.

The following testing levels are proposed.

7.1 Unit Testing

Unit tests verify individual functions.

Examples include:

- Battery percentage calculation.
- Energy consumption calculation.
- Vehicle-status validation.
- Route-distance calculation.
- Battery degradation prediction function.

Tool: PyTest

7.2 Integration Testing

Integration tests verify communication between different components.

Examples:

- Backend ↔ Database.
- Backend ↔ AI prediction module.
- Backend ↔ Charging optimization module.
- Backend ↔ Route optimization module.

The purpose is to ensure that individual modules work correctly when combined.

7.3 API Testing

API endpoints are tested to verify:

- Correct request handling.
- Correct response format.
- HTTP status codes.
- Invalid input handling.
- Authentication and authorization behavior where applicable.

Tool: Postman/Newman or automated Python API tests.

7.4 Acceptance Testing

After deployment to staging, acceptance tests verify whether the complete system behaves according to the expected requirements.

Example scenarios:

- Vehicle data is received successfully.
- Battery status is displayed correctly.
- Route recommendations are generated.
- Charging recommendations are generated.
- Fleet dashboard displays updated information.

Only if these tests pass should production deployment proceed.

8. Code Quality Analysis

The pipeline should include automated code-quality checks before deployment.

Recommended tool: SonarQube

SonarQube can be used to identify:

- Bugs.
- Code smells.
- Duplicated code.
- Maintainability problems.
- Reliability issues.

The pipeline should reject or stop deployment when predefined quality conditions are not satisfied.

Example Quality Gate

Build

↓

Tests Passed?

↓ YES

Code Quality Check

↓

Quality Gate Passed?

↓ YES

Continue Pipeline

This prevents low-quality code from reaching production.

9. Security Integration

Security checks are incorporated into the CI/CD pipeline to reduce the possibility of deploying vulnerable software.

9.1 Source-Code Security

Bandit can be used to identify common security problems in Python code.

Examples include:

- Unsafe function usage.
- Hard-coded security-sensitive information.
- Insecure coding patterns.

9.2 Dependency Security

Project dependencies should be scanned for known vulnerabilities.

Dependabot can be used to identify outdated or vulnerable dependencies and create update requests.

9.3 Secret Management

Sensitive information such as:

- Database passwords.
- API keys.
- Authentication credentials.
- Cloud credentials.

must not be stored directly in source code.

Instead, GitHub Actions Secrets or an equivalent secure secret-management mechanism should be used.

10. Docker Packaging

After all tests and quality checks pass, the EcoFleet AI application is packaged into a Docker container.

Advantages of Docker

- Provides a consistent runtime environment.
- Reduces environment-related deployment problems.
- Makes deployment easier.
- Supports scalability.
- Simplifies application version management.

A Docker image can be created after successful testing.

Source Code



Docker Build



EcoFleet AI Image



Image Registry

The image can then be stored in Docker Hub or GitHub Container Registry.

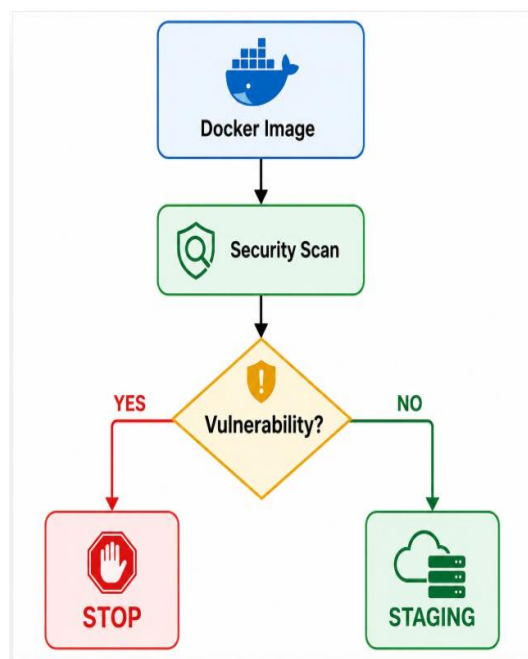
11. Container Image Security

Before deployment, the generated Docker image should be scanned for known vulnerabilities.

The image-scanning stage verifies:

- Vulnerable packages.
- Outdated dependencies.
- Known security issues.
- Unsafe base-image components.

If critical vulnerabilities are detected, the deployment should be stopped.



12. Deployment Strategy

The EcoFleet AI system uses three environments.

12.1 Development Environment

The development environment is used by developers to implement and test new features.

Typical activities:

- Coding.
- Debugging.
- Initial testing.
- Feature development.

12.2 Staging Environment

The staging environment represents the production environment as closely as practical.

The application is deployed here after:

- Build success.
- Unit tests.
- Integration tests.
- Code-quality checks.
- Security checks.
- Docker image creation.

Acceptance testing is performed in staging.

12.3 Production Environment

Production is the live environment used by fleet managers.

Only a successfully validated version should be promoted from staging to production.

Deployment Flow

Development



GitHub



CI Pipeline



Staging



Acceptance Testing



Approval / Successful Validation



Production

13. Deployment Strategy: Blue-Green Approach

A **blue-green deployment strategy** can be used to reduce production downtime.

Two environments are maintained:

- **Blue** – currently active version.
- **Green** – new version being deployed.

The new EcoFleet AI version is deployed to the green environment and tested.

If it works correctly:

Blue → Old Version

Green → New Version

Traffic



Green

If the new version fails:

Green → Failed

↓

Traffic

↓

Blue → Stable Version

This allows a quick rollback without rebuilding the previous version.

14. Monitoring and Notifications

After production deployment, the system must be continuously monitored.

Recommended tools

Prometheus

Used to collect system and application metrics.

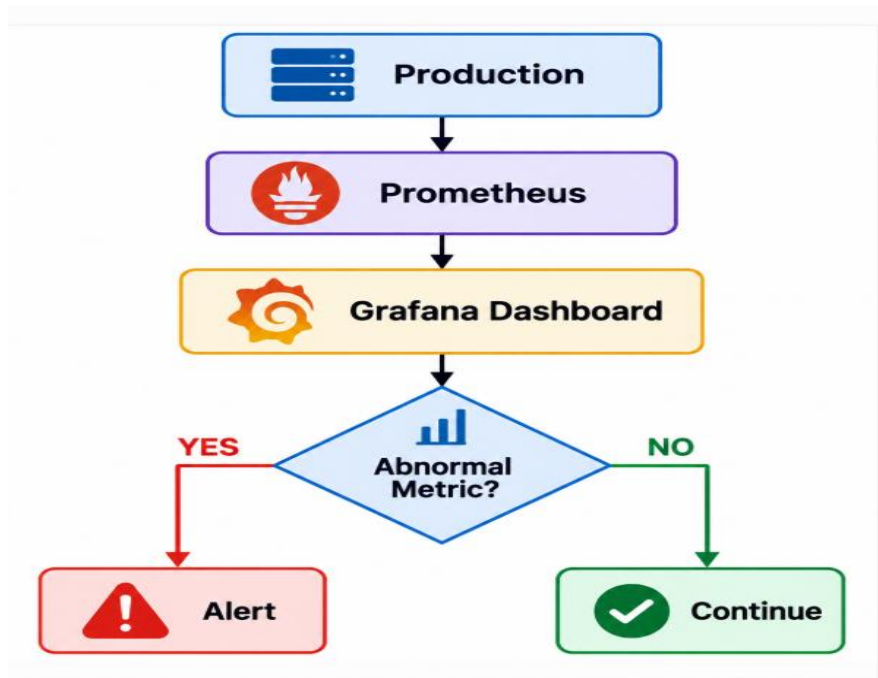
Grafana

Used to visualize metrics through dashboards.

Possible metrics include:

- API response time.
- Application uptime.
- CPU usage.
- Memory usage.
- Request count.
- Error rate.
- Database performance.

The monitoring system can alert the development or operations team when abnormal conditions occur.



15. Notification Mechanism

The CI/CD pipeline should provide notifications for important events.

Notifications may be generated when:

- Build fails.
- Tests fail.
- Security scan detects vulnerabilities.
- Quality gate fails.
- Staging deployment fails.
- Production deployment succeeds.
- Rollback is triggered.

This enables developers and administrators to respond quickly to pipeline failures.

16. Rollback Strategy

Rollback is necessary because a new production release may introduce unexpected errors.

The pipeline should maintain the previously successful Docker image.

Rollback Process

New Version Deployed



Production Monitoring



Problem Detected?



YES

NO



Rollback

Continue



Previous Stable Image



Production Restored

The previous stable Docker image can be redeployed if:

- Application errors increase.
- Critical functionality fails.
- Security problems are detected.
- Performance significantly decreases.
- Production acceptance checks fail.

The rollback mechanism improves system reliability and reduces downtime.

17. Complete CI/CD Workflow

The complete EcoFleet AI CI/CD workflow is:

Step 1 – Developer Commit

The developer implements a feature or fixes an existing problem and commits the changes to Git.

Step 2 – Push to GitHub

The code is pushed to the project repository.

Step 3 – CI Trigger

GitHub Actions automatically starts the pipeline.

Step 4 – Build

Dependencies are installed and the application is prepared.

Step 5 – Unit Testing

Individual functions and modules are tested using PyTest.

Step 6 – Integration Testing

Communication between application components is verified.

Step 7 – Code Quality

SonarQube performs code-quality analysis.

Step 8 – Security Testing

Bandit and dependency checks identify possible security issues.

Step 9 – Docker Packaging

A Docker image is generated.

Step 10 – Image Security Scan

The Docker image is checked for vulnerabilities.

Step 11 – Staging Deployment

The validated image is deployed to the staging environment.

Step 12 – Acceptance Testing

The complete application is tested against expected functionality.

Step 13 – Production Deployment

The validated version is deployed to production.

Step 14 – Monitoring

Prometheus and Grafana monitor application performance.

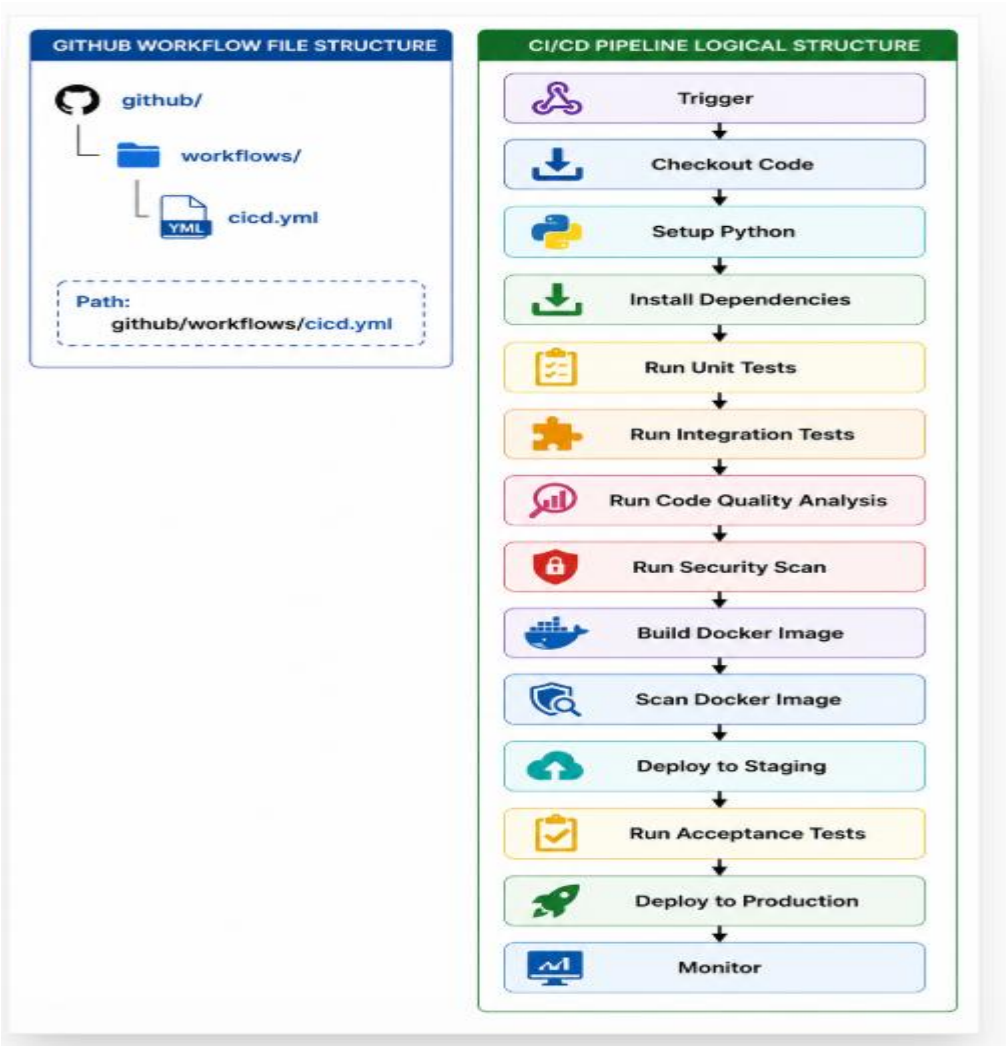
Step 15 – Rollback if Required If serious problems occur, the previous stable release is restored.

18. Tools and Technology Selection

Stage	Tool	Justification
Version Control	Git	Tracks source-code changes efficiently
Repository	GitHub	Supports collaboration and pull requests
CI/CD	GitHub Actions	Automates the complete pipeline
Backend	Python / FastAPI	Suitable for API and AI-based services
AI/ML	Scikit-learn	Supports predictive analytics
Testing	PyTest	Simple and effective Python testing framework
API Testing	Postman/Newman	Validates API functionality
Code Quality	SonarQube	Detects code-quality issues
Security	Bandit	Performs Python security analysis
Dependency Security	Dependabot	Identifies vulnerable dependencies
Containerization	Docker	Provides consistent deployment environments
Container Registry	GitHub Container Registry / Docker Hub	Stores versioned Docker images
Deployment	Docker Compose / Cloud VM	Simplifies multi-service deployment
Monitoring	Prometheus	Collects system/application metrics
Visualization	Grafana	Provides monitoring dashboards

19. CI/CD Pipeline Configuration Concept

The pipeline can be configured using a GitHub Actions workflow file such as:



20. Failure Handling

The pipeline should use a fail-fast approach for critical failures.

Failure	Pipeline Action
Build failure	Stop pipeline
Unit-test failure	Stop pipeline
Integration-test failure	Stop pipeline
Quality-gate failure	Stop pipeline
Critical security vulnerability	Stop pipeline

Docker build failure	Stop pipeline
Staging deployment failure	Stop pipeline
Acceptance-test failure	Prevent production deployment
Production failure	Trigger rollback
Monitoring alert	Notify responsible team

This ensures that defective software does not automatically reach production.

21. Benefits of the Proposed CI/CD Pipeline

The proposed CI/CD pipeline provides several benefits for EcoFleet AI:

- 1. Faster Software Delivery**
 Automated processes reduce manual deployment effort.
- 2. Improved Software Quality**
 Automated tests identify problems before deployment.
- 3. Better Security**
 Source-code and dependency security checks reduce vulnerabilities.
- 4. Consistent Deployment**
 Docker provides a standardized application environment.
- 5. Reduced Deployment Risk**
 Staging and acceptance testing prevent unvalidated versions from reaching production.
- 6. Quick Recovery**
 Rollback allows the previous stable version to be restored quickly.
- 7. Continuous Monitoring**
 Production metrics help identify failures and performance issues.
- 8. Improved Collaboration**
 GitHub and pull requests support structured team development.
- 9. Reduced Manual Errors**
 Automated build, testing, and deployment minimize human mistakes.
- 10. Scalability**
 The pipeline can support future expansion of EcoFleet AI services.

22. Security and Reliability Considerations

Security and reliability are important because EcoFleet AI processes fleet and vehicle information.

The proposed system therefore applies the following practices:

- Secure storage of credentials.
- No passwords or API keys inside source code.
- Dependency vulnerability scanning.
- Source-code security analysis.
- Docker image scanning.
- Pull-request-based code review.
- Automated testing before deployment.
- Staging validation.
- Production monitoring.
- Versioned releases.
- Rollback to a known stable version.

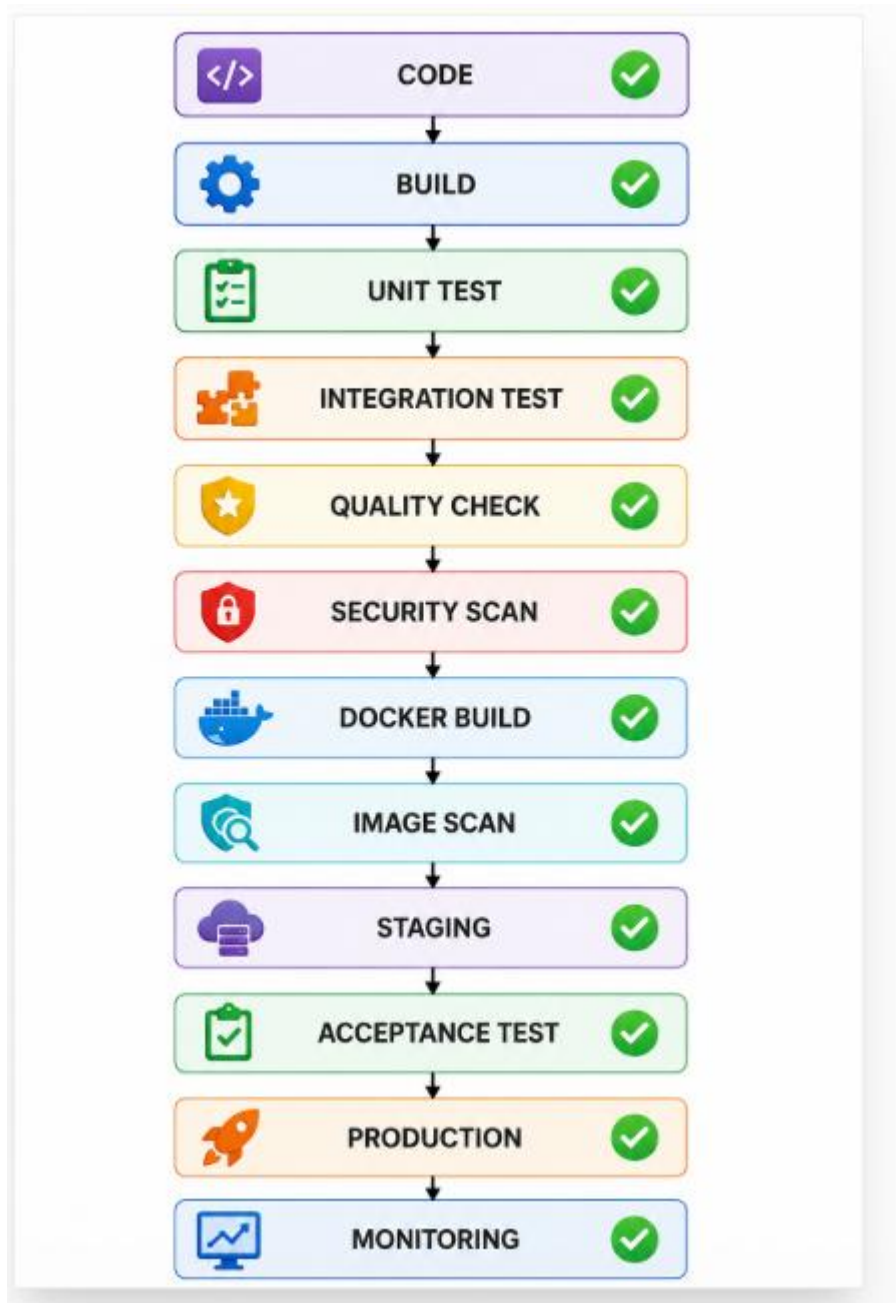
These practices help protect the application and improve operational reliability.

23. Expected CI/CD Pipeline Output

After successful execution, the pipeline should produce:

- Successfully tested source code.
- Quality-approved application.
- Security-checked dependencies.
- Versioned Docker image.
- Successfully deployed staging application.
- Passed acceptance tests.
- Production deployment.
- Monitoring dashboard.
- Deployment status notification.

A successful pipeline can therefore be represented as:



24. Conclusion

The proposed CI/CD pipeline for EcoFleet AI: Intelligent Electric Vehicle Fleet Monitoring and Route Optimization Platform provides an automated and structured approach for developing, testing, securing, packaging, and deploying the system.

The pipeline integrates GitHub and GitHub Actions for source management and automation, pytest for automated testing, SonarQube for code-quality analysis, Bandit and dependency scanning for security, Docker for containerized packaging, and Prometheus and Grafana for production monitoring.

The use of development, staging, and production environments ensures that software is validated before reaching users. A rollback mechanism further improves reliability by allowing the previous stable version to be restored when a production problem occurs.

Overall, the proposed CI/CD architecture supports continuous integration, automated quality assurance, secure deployment, continuous monitoring, and reliable software delivery for the EcoFleet AI platform. It provides a scalable foundation that can accommodate future improvements in AI-based battery prediction, charging optimization, route planning, and fleet analytics.